

Ejercicio 3

¿Hay algún error en el siguiente programa? Si es así, explique por qué y corrija.

```
1  #include <iostream>
2  class C
3  {
4      public:
5          C(int i = 0, bool al = false): n{i}, hayAlerta{al} {}
6          void mostrar()
7          {
8              if (hayAlerta)
9                  alerta();
10             std::cout << "i = " << n << std::endl;
11         }
12     private:
13         int n;
14         bool hayAlerta;
15         void alerta() { std::cout << "Alerta " << std::endl; }
16 };
17
18 int main()
19 {
20     const C c;
21     c.mostrar();
22 }
```

El error se produce porque el objeto `c` de `C` está declarado constante y está llamando un método de la clase `C` que no tiene comportamiento definido para objetos constantes:

Solución:

```
1  void mostrar() const {
2      if (hayAlerta)
3          alerta();
4      std::cout << "i = " << n << std::endl;
5  }
6
7  void alerta() const { std::cout << "Alerta " <<
8      std::endl; }
```

Ejercicio 4

Indique los errores que hay en el código e indique el motivo.

```
1  class C {
2  public:
3      C();
4      C(int a, int b, int c, int d);
5      int f1(int i) const;
6      int f2(int i);
7      static void f3() { m = h; }
8      static int n;
9
10 private:
11     int h;
12     mutable int i;
13     const int j;
14     mutable int k;
15     static int m;
16 };
17
18 int C::f1(int i) const
19 {
20     h = i;
21     k = i;
22     return 0;
23 }
24
25 C::C() { h = i = j = k = 0; }
26
27 C::C(int a, int b, int c, int d) : h(a), i(b), j(c), k(d) {}
28
29 int C::f2(int i)
30 {
31     k = i;
32     h = i;
33     return 0;
34 }
```

El primer problema es que el método `f1` es `const` y está modificando un atributo que no es mutable en este caso `h`;

Segundo problema se encuentra en el constructor que se inicializa incorrectamente el atributo constante `j` que debe inicializarse en la lista de inicialización del constructor.

El tercer problema está en el método `f3()`, ya que es un método `static` e intenta acceder directamente al atributo `h`. Como `h` es un atributo de instancia, necesita un objeto concreto para poder ser accedido. Al ser `static`, `f3()` no tiene puntero `this`, por lo que no puede referirse directamente a `h`.

Ejercicio 5

Considerando el código de la pregunta anterior, determine si hay errores en el código de esta pregunta. ¿Falta algo para que podamos llegar a ejecutar este programa? En tal caso, haga las correcciones necesarias para solucionarlo e indique qué imprime el programa.

```
1  #include <iostream>
2
3  class C { // clase C anterior
4      // ...
5  };
6
7  // definición de métodos de la clase C aquí...
8
9  int main()
10 {
11     C c1, c2;
12     C::n = 3;
13     c1.n = 4;
14     std::cout << C::n << "_" << c2.n << std::endl;
15 }
```

Este código no está correcto porque necesitamos definir el valor de static int n.

Cambios

```
1  class C { // clase C anterior
2      // ...
3  };
4
5  // definición de métodos de la clase C aquí...
6
7  int C::n{/* valor inicializar */};
8
9  int main()
10 {
11     C c1, c2;
12     C::n = 3;
13     c1.n = 4;
14     std::cout << C::n << "_" << c2.n << std::endl;
15 }
```

Ejercicio 6

Escriba las declaraciones correspondientes a la sobrecarga del operador + para objetos de tipo C como miembro y como función externa. A continuación, escriba a modo de ejemplo un trozo de código que provoque un error de compilación con el operador miembro, pero no con el externo.

```
1  class C
2  {
3      public:
4          C(int i) : n(i), m(0.0) {} // ctor. de conversión
5          // ...
```

```

6
7     C operator +(const C& B) const;
8
9     friend C operator +(const C& A, const C& B);
10
11 private:
12     int n;
13     double m;
14 };
15
16 C operator +(const C& A, const C& B)
17 {
18     C c(A.n);
19
20     c.n = A.n + B.n;
21     c.m = A.m + B.m;
22
23     return c;
24 }
25
26 C C::operator +(const C& B) const
27 {
28     C c(this->n); //Para evitar error al no tener ctor defect
29
30     c.n = this->n + B.n;
31     c.m = this->m + B.m;
32
33     return c;
34 }
35
36 int main()
37 {
38     C a(1);
39
40     C resultado = 1 + a; //Error con el miembro ya que no hace conversión del primer
                           //operando, pero funciona con el externo
41
42     resultado = a + 1; //No da error con ninguno
43 }

```